



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)

НАПРАВЛЕНИЕ ПОДГОТОВКИ 09.04.01 Информатика и вычислительная техника

МАГИСТЕРСКАЯ ПРОГРАММА 09.04.01/07 Интеллектуальные системы анализа,
обработки и интерпретации больших данных

О Т Ч Е Т

по лабораторной работе № 4

Название: система планирования «толстый» клиент-сервер

Дисциплина: информационная безопасность в системах поддержки принятия
решений

Студент

ИУ6-42М

(Группа)

(Подпись, дата)

Д.А. Залимханов

(И.О. Фамилия)

Преподаватель

(Подпись, дата)

М.А. Миков

(И.О. Фамилия)

Москва, 2025

Цель: построение распределенной двухуровневой системы управления

Задание: разработать приложение, у которого:

- учетные записи «Логин — пароль» имеют вид: администратор admin — 123; диспетчер — operator — 321; начальники цехов — foreman1 — 456, foreman2 — 678, foreman3 — 890;
- администратор может изменять существующий набор данных (таблицу) по нажатию кнопки «Редактирование датасета»;
- с помощью данного функционала он может назначить уникальное имя таблице и закрепить её за одним из начальников цехов;
- администратор может самостоятельно осуществить введение таблицы или просто сохранить её в системе для дальнейшего решения.

На рисунке 1 показана структура приложения толстый клиент.

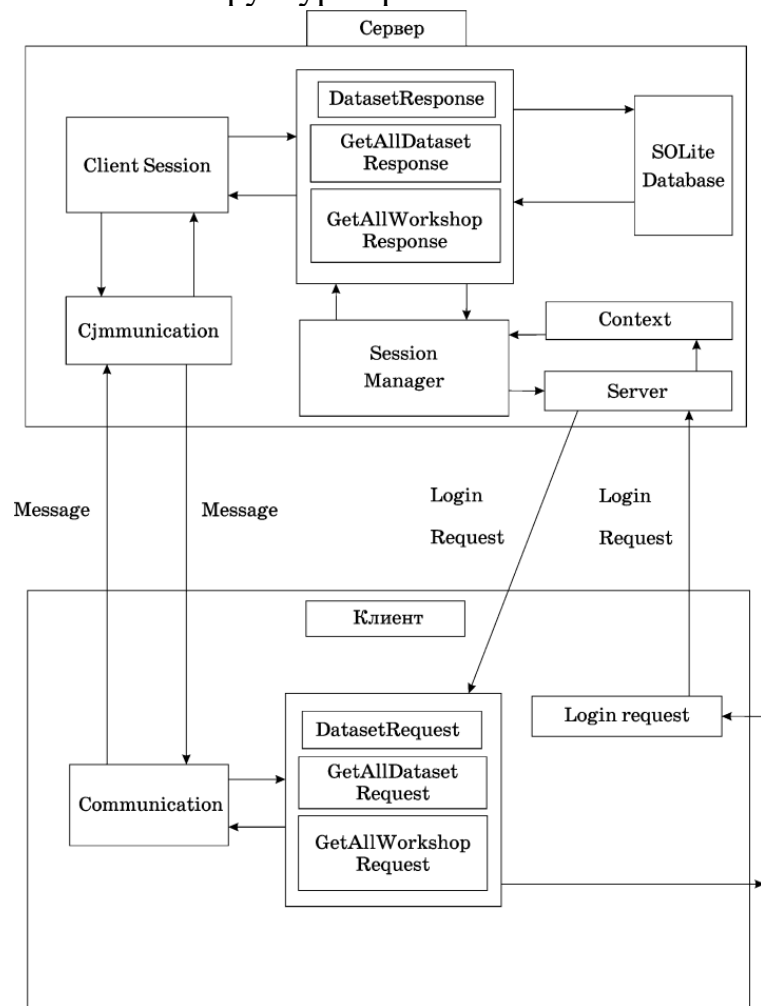


Рисунок 1 – структура приложения

На рисунке 2 показана схема базы данных.

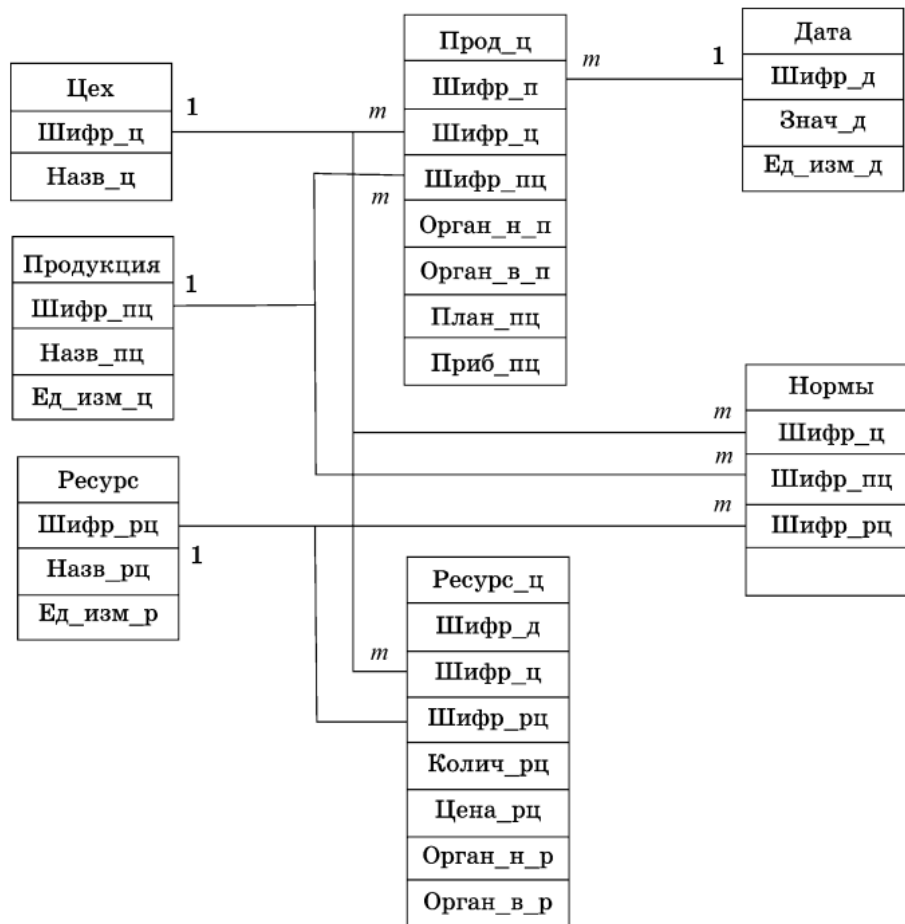


Рис. 4.3
Схема связей базы
данных disp уровня
 $h = 2$

Рисунок 2 – Схема базы данных

Код компонента DatasetDetails.js:

```

import React, { useState, useEffect } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import api from '../utils/api';
import { useAuth } from '../contexts/AuthContext';

const DatasetDetails = () => {
  const { id } = useParams();
  const navigate = useNavigate();
  const { user, isAdmin, isDispatcher, isWorkshopManager, addAuthToRequest } =
    useAuth();

  const [dataset, setDataset] = useState(null);
  const [workshops, setWorkshops] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [isEditing, setIsEditing] = useState(false);
  const [editedDataset, setEditedDataset] = useState(null);
  const [saveAsNew, setSaveAsNew] = useState(false);
  const [newName, setNewName] = useState('');

  useEffect(() => {
    const fetchData = async () => {
      try {
    
```

```

        setLoading(true);

        // Получаем информацию о датасете
        const datasetResponse = await api.post(`/datasets/${id}`,
addAuthToRequest());
        setDataset(datasetResponse.data);

        // Устанавливаем начальные данные для редактирования
        setEditedDataset(datasetResponse.data);
        setNewName(datasetResponse.data.name);

        // Получаем список цехов
        const workshopsResponse = await api.post('/workshops', addAuthToRequest());
        setWorkshops(workshopsResponse.data);

        setError('');
    } catch (error) {
        console.error('Ошибка при загрузке данных:', error);
        setError('Произошла ошибка при загрузке данных');
    } finally {
        setLoading(false);
    }
};

fetchData();
}, [id, addAuthToRequest]);

const handleSave = async () => {
    try {
        setLoading(true);

        const dataToSend = {
            name: newName || editedDataset.name,
            M: editedDataset.matrix.length,
            N: editedDataset.matrix[0]?.length || 0,
            matrix: editedDataset.matrix,
            workshop: editedDataset.workshop
        };

        let response;

        if (saveAsNew) {
            // Сохраняем как новый датасет
            response = await api.post('/datasets/create', addAuthToRequest(dataToSend));
            navigate(`/datasets/${response.data.id}`);
        } else {
            // Обновляем существующий датасет
            response = await api.post(`/datasets/update/${id}`,
addAuthToRequest(dataToSend));
            setDataset(response.data);
        }
    }

```

```

        setIsEditing(false);
        setSaveAsNew(false);
        setError('');
    } catch (error) {
        console.error('Ошибка при сохранении датасета:', error);
        setError('Произошла ошибка при сохранении датасета');
    } finally {
        setLoading(false);
    }
};

const handleDelete = async () => {
    if (!window.confirm('Вы действительно хотите удалить этот датасет?')) {
        return;
    }

    try {
        setLoading(true);

        await api.post(`/datasets/delete/${id}`, addAuthToRequest());

        navigate('/datasets');
    } catch (error) {
        console.error('Ошибка при удалении датасета:', error);
        setError('Произошла ошибка при удалении датасета');
    }
};

const handleMatrixChange = (rowIndex, colIndex, value) => {
    const newMatrix = [...editedDataset.matrix];
    newMatrix[rowIndex][colIndex] = parseInt(value) || 0;

    setEditedDataset({
        ...editedDataset,
        matrix: newMatrix
    });
};

if (loading) {
    return (
        <div className="flex justify-center items-center h-64">
            <div className="text-lg text-gray-500">Загрузка...</div>
        </div>
    );
}

if (error) {
    return (
        <div className="bg-red-100 border border-red-400 text-red-700 px-4 py-3 rounded
mb-4">
            {error}
        </div>
    );
}

```

```

    );
  }

  if (!dataset) {
    return (
      <div className="bg-yellow-100 border border-yellow-400 text-yellow-700 px-4 py-3 rounded mb-4">
        Датасет не найден
      </div>
    );
  }

  const canEdit = isAdmin || (isDispatcher && isEditing) || (isWorkshopManager && dataset.workshop === user.workshop);

  return (
    <div className="space-y-8">
      <div className="flex justify-between items-center">
        <h1 className="text-2xl font-bold text-gray-800">
          {isEditing ? 'Редактирование датасета' : 'Просмотр датасета'}
        </h1>

        <div className="flex space-x-3">
          {!isEditing && canEdit && (
            <button
              onClick={() => setIsEditing(true)}
              className="px-4 py-2 bg-indigo-600 text-white rounded-md hover:bg-indigo-700 focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-indigo-500"
            >
              Редактировать
            </button>
          )}

          <button
            onClick={() => navigate('/datasets')}
            className="px-4 py-2 border border-gray-300 text-gray-700 rounded-md hover:bg-gray-50 focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-indigo-500"
          >
            Назад к списку
          </button>
        </div>
      </div>

      <div className="bg-white shadow rounded-lg p-6">
        {isEditing ? (
          <div className="space-y-6">
            {(isAdmin || (isWorkshopManager && saveAsNew)) && (
              <div>
                <label htmlFor="name" className="block text-sm font-medium text-gray-700">
                  Название

```

```

        </label>
        <input
          type="text"
          id="name"
          value={isWorkshopManager && saveAsNew ? newName :
editedDataset.name}
          onChange={(e) => {
            if (isWorkshopManager && saveAsNew) {
              setNewName(e.target.value);
            } else {
              setEditedDataset({ ...editedDataset, name: e.target.value });
            }
          }}
          className="mt-1 block w-full border border-gray-300 rounded-md
shadow-sm py-2 px-3 focus:outline-none focus:ring-indigo-500 focus:border-indigo-500"
        />
      </div>
    )}

    {isWorkshopManager && (
      <div className="bg-yellow-50 border border-yellow-100 p-4 rounded-md">
        <div className="flex items-center mb-4">
          <input
            type="checkbox"
            id="saveAsNew"
            checked={saveAsNew}
            onChange={(e) => setSaveAsNew(e.target.checked)}
            className="h-4 w-4 text-indigo-600 focus:ring-indigo-500 border-
gray-300 rounded"
          />
          <label htmlFor="saveAsNew" className="ml-2 block text-sm text-gray-
900">
            Сохранить как новый датасет
          </label>
        </div>
        {saveAsNew && (
          <p className="text-sm text-yellow-700">
            Данные будут сохранены в новый датасет. Исходный датасет
останется без изменений.
          </p>
        )}
      </div>
    )}

    {(isAdmin || isDispatcher) && (
      <div>
        <label htmlFor="workshop" className="block text-sm font-medium text-
gray-700">
          Цех
        </label>
        <select
          id="workshop"

```

```

        value={editedDataset.workshop || ''}
        onChange={(e) => setEditedDataset({
            ...editedDataset,
            workshop: e.target.value === '' ? null : parseInt(e.target.value)
        })}
        className="mt-1 block w-full border border-gray-300 rounded-md
shadow-sm py-2 px-3 focus:outline-none focus:ring-indigo-500 focus:border-indigo-500"
    >
        <option value="">Не выбрано</option>
        {workshops.map(workshop => (
            <option key={workshop.id} value={workshop.id}>
                {workshop.name || `Цех #${workshop.id}`}
            </option>
        ))}
    </select>
</div>
)}

{(isAdmin || isWorkshopManager) && (
    <div>
        <label className="block text-sm font-medium text-gray-700 mb-2">
            Матрица данных
        </label>
        <div className="overflow-x-auto">
            <table className="min-w-full divide-y divide-gray-200 border">
                <tbody className="bg-white divide-y divide-gray-200">
                    {editedDataset.matrix?.map((row, rowIndex) => (
                        <tr key={rowIndex}>
                            {row.map((cell, colIndex) => (
                                <td key={colIndex} className="px-2 py-2 whitespace-nowrap
border">
                                    <input
                                        type="number"
                                        value={cell}
                                        onChange={(e) => handleMatrixChange(rowIndex,
colIndex, e.target.value)}
                                        className="block w-full border-0 p-1 focus:outline-
none focus:ring-0 text-center"
                                        style={{ width: '60px' }}
                                    />
                                </td>
                            ))}
                        </tr>
                    ))}
                </tbody>
            </table>
        </div>
    </div>
)}

<div className="flex justify-end space-x-3 pt-4">
    <button

```

```

        onClick={() => setIsEditing(false)}
        className="px-4 py-2 border border-gray-300 text-gray-700 rounded-md
hover:bg-gray-50 focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-
indigo-500"
      >
        Отмена
      </button>

      <button
        onClick={handleSave}
        disabled={loading}
        className="px-4 py-2 bg-indigo-600 text-white rounded-md hover:bg-
indigo-700 focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-indigo-500"
      >
        {loading ? 'Сохранение...' : saveAsNew ? 'Сохранить как новый' :
'Sохранить изменения'}
      </button>
    </div>
  </div>
) : (
  <div className="space-y-6">
    <div>
      <h2 className="text-xl font-semibold text-gray-800 mb-2">{dataset.name
|| `Датасет #${dataset.id}`}</h2>

      <div className="mt-4 grid grid-cols-2 gap-4">
        <div>
          <p className="text-sm font-medium text-gray-500">ID:</p>
          <p className="mt-1">{dataset.id}</p>
        </div>

        <div>
          <p className="text-sm font-medium text-gray-500">Размер:</p>
          <p className="mt-1">{dataset.M && dataset.N ?
`${dataset.M}x${dataset.N}` : 'Не указан'}</p>
        </div>

        <div>
          <p className="text-sm font-medium text-gray-500">Цех:</p>
          <p className="mt-1">
            {dataset.workshop ? (
              workshops.find(w => w.id === dataset.workshop)?.name || `Цех
#${dataset.workshop}`
            ) : (
              'Не назначен'
            )}
          </p>
        </div>
      </div>
    </div>
  </div>
</div>

```

```

        <h3 className="text-lg font-medium text-gray-800 mb-3">Матрица
данных</h3>

        {dataset.matrix ? (
          <div className="overflow-x-auto">
            <table className="min-w-full divide-y divide-gray-200 border">
              <tbody className="bg-white divide-y divide-gray-200">
                {dataset.matrix.map((row, rowIndex) => (
                  <tr key={rowIndex}>
                    {row.map((cell, colIndex) => (
                      <td key={colIndex} className="px-4 py-2 whitespace-nowrap
text-center border">
                        {cell}
                      </td>
                    ))}
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        ) : (
          <p className="text-gray-500">Данные отсутствуют</p>
        )}
      </div>
    </div>
  )}
</div>
</div>
);
};

export default DatasetDetails;

```

Код сервера:

```

const express = require('express');
const router = express.Router();
const { authMiddleware, adminMiddleware, dispatcherMiddleware } =
require('../middleware/auth');
const db = require('../db');

// Хранилище датасетов в памяти (аналог HashMap в Java-примере)
const datasets = {
  'Dataset 1': generateRandomDataset(10, 8),
  'Dataset 2': generateRandomDataset(10, 8),
  'Dataset 3': generateRandomDataset(10, 8)
};

// Хранилище для соответствия датасетов и цехов
const datasetWorkshops = {
  'Dataset 1': null,
  'Dataset 2': null,
  'Dataset 3': null
};

```

```

// Функция для генерации случайных данных для датасета
function generateRandomDataset(rows, cols) {
  const data = [];
  for (let i = 0; i < rows; i++) {
    const row = [];
    row.push(`D${i + 1}`); // Прод
    row.push(`i${Math.floor(Math.random() * 26)}`); // Шифр_п
    row.push(`j${Math.floor(Math.random() * 26)}`); // Шифр_ц
    row.push(`c${Math.floor(Math.random() * 26)}`); // Шифр_пц

    // Орган_н_п
    row.push(String.fromCharCode(97 + Math.floor(Math.random() * 26)));

    // Орган_в_п
    row.push(String.fromCharCode(97 + Math.floor(Math.random() * 26)));

    // План_пц
    row.push(Math.floor(Math.random() * 100));

    // Приб_пц
    row.push(Math.floor(Math.random() * 100));

    data.push(row);
  }
  return data;
}

// Получение списка всех датасетов
router.post('/list', authMiddleware, (req, res) => {
  res.json({
    datasets: Object.keys(datasets)
  });
});

// Получение данных конкретного датасета
router.post('/get', authMiddleware, (req, res) => {
  const { name } = req.body;

  if (!name || !datasets[name]) {
    return res.status(404).json({ message: 'Датасет не найден' });
  }

  res.json({
    name: name,
    data: datasets[name],
    workshopId: datasetWorkshops[name]
  });
});

// Сохранение данных датасета
router.post('/save', authMiddleware, (req, res) => {

```

```

const { name, data } = req.body;

if (!name || !data) {
  return res.status(400).json({ message: 'Неверные данные для сохранения' });
}

datasets[name] = data;

res.json({
  message: 'Датасет успешно сохранен',
  name: name
});
});

// Обновление датасета (переименование и привязка к цеху)
router.post('/update', authMiddleware, dispatcherMiddleware, (req, res) => {
  const { oldName, newName, workshopId } = req.body;

  if (!oldName || !newName || !datasets[oldName]) {
    return res.status(400).json({ message: 'Неверные данные для обновления датасета'
});
  }

  // Проверяем существование цеха, если указан ID
  if (workshopId) {
    try {
      const workshop = db.prepare('SELECT * FROM workshops WHERE id =
?').get(workshopId);
      if (!workshop) {
        return res.status(404).json({ message: 'Указанный цех не найден' });
      }
    } catch (error) {
      console.error('Ошибка при проверке цеха:', error);
      return res.status(500).json({ message: 'Ошибка сервера при обновлении датасета'
});
    }
  }

  // Переименовываем датасет, если имя изменилось
  if (oldName !== newName) {
    datasets[newName] = datasets[oldName];
    delete datasets[oldName];
  }

  // Обновляем привязку к цеху
  datasetWorkshops[newName] = workshopId || null;

  res.json({
    message: 'Датасет успешно обновлен',
    name: newName,
    workshopId: datasetWorkshops[newName]
  });
});

```

```

});

// Переименование датасета
router.post('/rename', authMiddleware, dispatcherMiddleware, (req, res) => {
  const { oldName, newName } = req.body;

  if (!oldName || !newName || !datasets[oldName]) {
    return res.status(400).json({ message: 'Неверные данные для переименования' });
  }

  datasets[newName] = datasets[oldName];
  datasetWorkshops[newName] = datasetWorkshops[oldName];
  delete datasets[oldName];
  delete datasetWorkshops[oldName];

  res.json({
    message: 'Датасет успешно переименован',
    name: newName
  });
});

// Создание нового датасета (доступно только для администратора)
router.post('/create', authMiddleware, adminMiddleware, (req, res) => {
  const { rows, cols } = req.body;

  if (!rows || !cols || isNaN(rows) || isNaN(cols)) {
    return res.status(400).json({ message: 'Неверные данные для создания датасета' });
  }

  const newName = `Dataset ${Object.keys(datasets).length + 1}`;
  datasets[newName] = generateRandomDataset(parseInt(rows), parseInt(cols));
  datasetWorkshops[newName] = null;

  res.json({
    message: 'Датасет успешно создан',
    name: newName
  });
});

module.exports = router;

```

Код базы данных:

```

const initSQL = `
BEGIN TRANSACTION;
DROP TABLE IF EXISTS workshops;
CREATE TABLE IF NOT EXISTS workshops (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT
);

DROP TABLE IF EXISTS users;
CREATE TABLE IF NOT EXISTS users (

```

```

    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL UNIQUE,
    password TEXT NOT NULL,
    role INTEGER NOT NULL DEFAULT 0,
    workshop INTEGER,
    FOREIGN KEY(workshop) REFERENCES workshops(id)
);
INSERT INTO users VALUES (1, 'admin', '123', 2, NULL);

DROP TABLE IF EXISTS datasets;
CREATE TABLE IF NOT EXISTS datasets (
    id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    M INTEGER,
    N INTEGER,
    data TEXT,
    workshop INTEGER,
    FOREIGN KEY(workshop) REFERENCES workshops(id)
);
INSERT INTO datasets VALUES (1, 'test1', 3, 2, '1;2;3;4;5;6', NULL);
INSERT INTO datasets VALUES (2, 'test2', NULL, 4, NULL, NULL);

-- Добавим тестовых пользователей разных ролей
INSERT INTO workshops VALUES (1, 'Цех №1');
INSERT INTO workshops VALUES (2, 'Цех №2');

-- Добавим диспетчера
INSERT INTO users VALUES (2, 'dispatcher', '123', 1, NULL);

-- Добавим начальников цехов
INSERT INTO users VALUES (3, 'workshop1', '123', 0, 1);
INSERT INTO users VALUES (4, 'workshop2', '123', 0, 2);

COMMIT;
`;

```

На рисунках 3 – 10 показана работа приложения.

```

> workshop-management-server@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.10
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
Сервер запущен на порту 5000

```

Рисунок 3 – работа сервера

Вход в систему

Имя пользователя:

Пароль:

Войти
Выход

Доступные учетные записи:

- admin / 123 (Администратор)
- operator / 321 (Диспетчер)
- foreman1 / 456 (Начальник цеха 1)
- foreman2 / 678 (Начальник цеха 2)
- foreman3 / 890 (Начальник цеха 3)

Рисунок 4 – Авторизация

Управление датасетами							Вы вошли как: admin
Выберите датасет:		Выбранный датасет: Dataset 1					
Dataset 1							
ПРОД	ШИФР_П	ШИФР_Ц	ШИФР_ПЦ	ОРГАН_Н_П	ОРГАН_В_П	ПЛАН_ПЦ	ПРИБ_ПЦ
D1	i16	j2	c25	j	l	6	60
D2	i12	j18	c23	o	e	7	7
D3	i0	j11	c24	q	x	24	50
D4	i13	j13	c4	i	e	11	24
D5	i25	j17	c12	k	k	32	3
D6	i22	j6	c17	g	l	34	87
D7	i8	j5	c21	t	s	62	45
D8	i3	j8	c23	f	a	58	96
D9	i15	j1	c23	y	a	81	18
D10	i3	j11	c15	q	a	80	45
Сохранить Редактировать Добавить Рассчитать							

Рисунок 5 – Вывод датасета

Результаты расчетов

CX: 3911657.90

C: 2.90, 4.10, 4.30, 3.50, 3.50, 3.80, 240410.80, 278588.00, 0.00, 0.00

DN: 1.00, 2.00, 3.00, 1.00, 1.00, 1.00, 1.00, 1.00, 0.00, 0.00

DV: 8.00, 8.00, 8.00, 8.00, 8.00, 6.00, 7.00, 8.00, 0.00, 0.00

BN: 105.00, 105.00, 105.00, 269.00, 248.00, 105.00, 105.00, 105.00, 105.00, 105.00

BV: 1174.00, 1174.00, 1174.00, 1174.00, 1174.00, 934.00, 1174.00, 1174.00, 1174.00, 920.00

X: 1.00, 2.00, 3.00, 4.00, 5.00, 6.00, 7.00, 8.00, 0.00, 0.00

AX: 522.00, 105.00, 568.00, 269.00, 248.00, 934.00, 794.00, 1174.00, 711.00, 920.00

Рисунок 6 – Результаты работы генератора данных

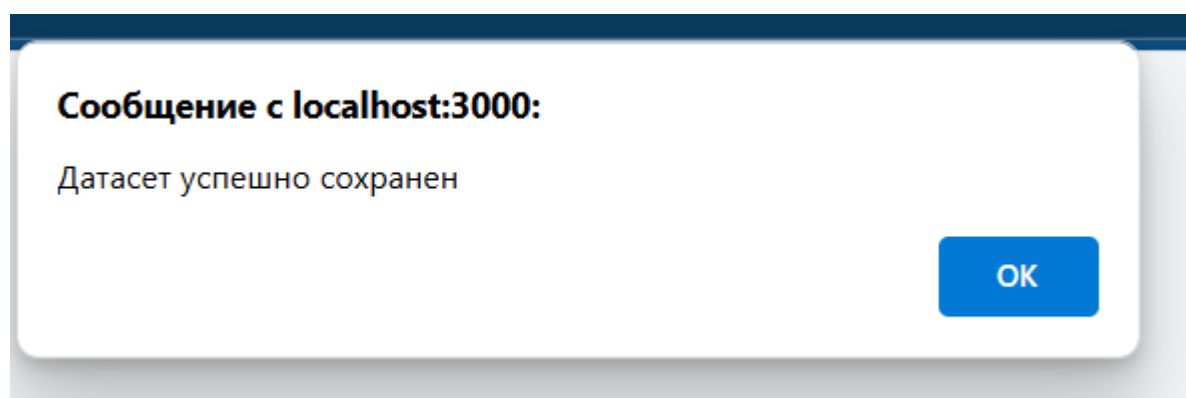


Рисунок 7 – Сохранение таблицы

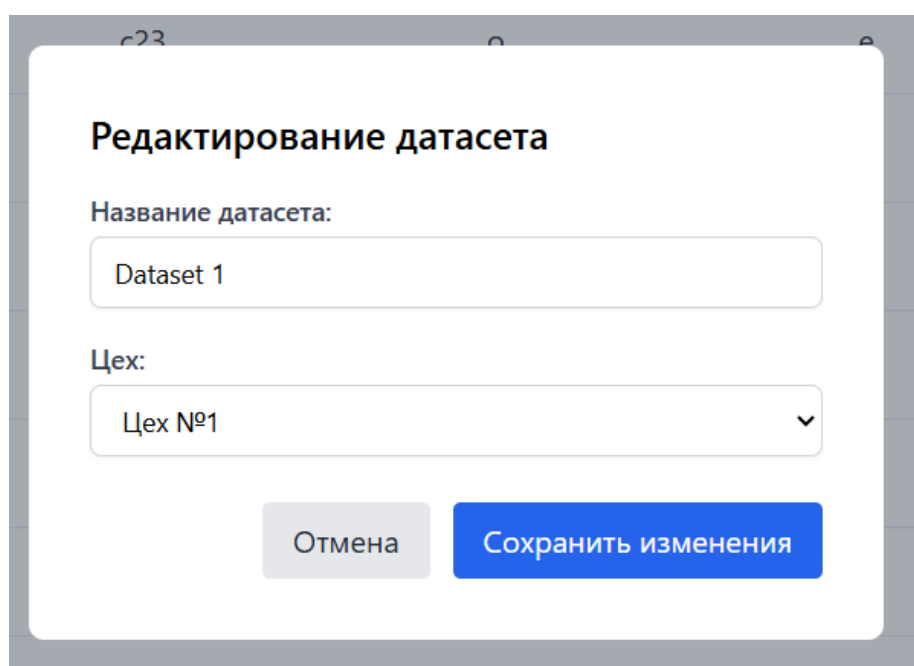


Рисунок 8 – Изменение таблицы

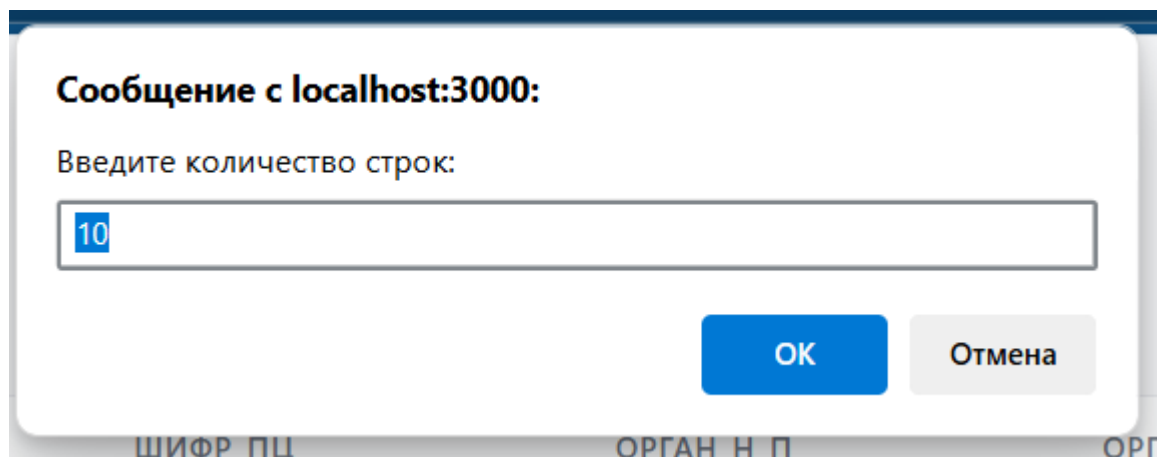


Рисунок 9 – Создание новой таблицы

Вывод: в процессе выполнения лабораторной работы была выполнена разработка толстого клиента.